

Deep Drift: saved versions and how git works

A guide for Hasib. Keeps track of every saved version of the dive, and explains how to go back to one.
Updated 24 Sep 2026.

In one sentence: git is a program that takes a photo of the whole project each time we save (a *commit*), keeps every photo forever, and can put any old photo back. Before each realism change we have a safe copy to return to.

1. Where everything is

Git program	<code>C:\Program Files\Git\cmd\git.exe</code> (the one Windows uses) <code>C:\Program Files\Git\mingw64\bin\git.exe</code> (the same program, used inside the Git Bash window)
Git version	git version 2.54.0.windows.1
The project	<code>C:\Users\hasib\Downloads\deep-drift-project</code>
The history	<code>C:\Users\hasib\Downloads\deep-drift-project\.git</code> A hidden folder, about 1.2 MB. Every saved version lives inside it (packed and compressed). Never edit or delete it: deleting it deletes all history. Your normal files are not affected.
Settings for this project	<code>.git\config</code> . Name <code>Hasib</code> , email <code>hasibahmad.njupt@gmail.com</code> . These are set for this project only. Git on this computer has no global name or email.
Branch	<code>main</code> (the main line of work; see section 3)
Tags (named versions)	<code>v1.0</code>
Online copy	None. The history exists only on this computer. If the project folder is lost, the history is lost too. Putting a copy on GitHub (private) would fix that; ask when you want it.
Plain backup page	<code>backup\deep-drift-v1.0.html</code> . Double-click it to play the v1.0 dive. No git needed.
This guide	<code>docs\git-tracking.pdf</code> , made by <code>docs\make_git_tracking.py</code>
Special files	<code>.gitignore</code> : things git ignores (<code>node_modules</code> , screenshots <code>*.png</code> , <code>__pycache__</code>). <code>.gitattributes</code> : keeps line endings exactly as saved. Without it, Windows git would switch them when restoring, so an old version would not come back byte for byte.

2. Why the version is called v1.0, and how numbers go up

The project file `package.json` said `0.4.0` before git was set up. A number starting with 0 usually means "still being built". You chose to call the current dive **v1.0**: the first saved version. `package.json` was changed to

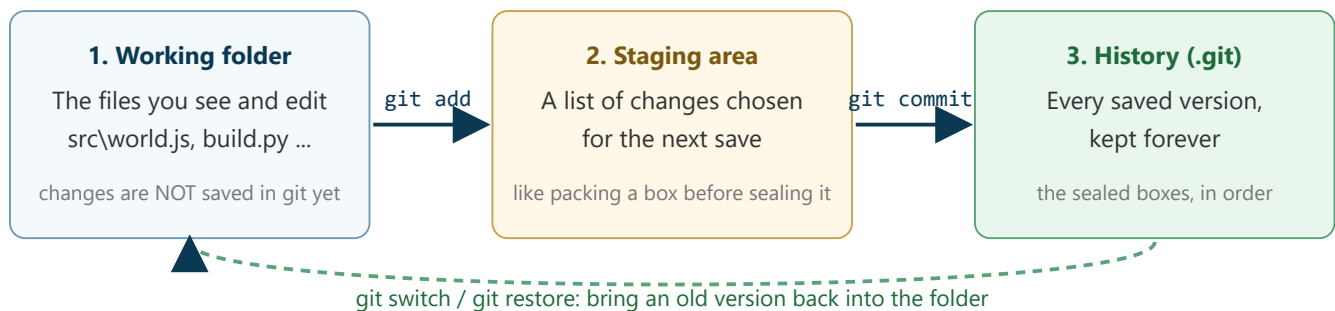
1.0.0 so everything agrees.

The rule from now on (**vMAJOR.MINOR**):

- **MINOR goes up by one** for each finished, tested change to the dive: v1.1 (reef wall), v1.2 (wreck), and so on. After v1.9 comes v1.10, not v2.0.
- **MAJOR goes up** (v2.0) only for a very big change you decide on, for example a full redesign.
- **No new number** for commits that do not change the dive: this guide, test fixes, notes. They still get a row in the table, with "none" as the version.
- If a version is undone, its number is **not reused**. The next change takes the next number, so a number always means one thing.

3. How git works

The three places a change passes through



1. **Working folder.** The files as they are now. When code is edited, only this changes. Git notices, but nothing is saved yet.
2. **Staging area.** `git add` puts chosen changes here. It is a waiting list for the next save, so one save can hold one tidy change even if other files were also touched.
3. **History.** `git commit` seals everything on the waiting list into a new saved version inside `.git`. After that it is safe: later edits cannot damage it.

What a commit is

A commit is a complete snapshot of every tracked file at that moment, plus a message (what and why), the author, the date, and a pointer to the commit before it. Git stores only what changed, so saving often is cheap: the whole history today is about 1.2 MB.

Each commit gets an **ID** like `7929f37` (short form) or a 40-character long form. The ID is a fingerprint calculated from the commit's content. Change one letter anywhere and the ID changes completely. That is why an old version cannot be changed quietly. It is also why this file cannot show the ID of the commit that contains it: the ID is only known after the file is saved. That row is filled in at the next update.

History, branch, tag and HEAD



Each commit points back to the one before it. "main" moves forward with every new commit; a tag never moves.

- **History** is a chain: every commit points back to its parent.
- **Branch** (`main`): a name that points at the newest commit and moves forward with every commit. We use only one branch.
- **Tag** (`v1.0`): a permanent name for one commit. It never moves. This is how "go back to v1.1" stays easy.
- **HEAD**: "where you are now", the version the working folder is based on. Normally HEAD is on `main`.

4. How each change will be made

1. Edit the source files in `src\` (never `dist\index.html` by hand).
2. Rebuild: `python build.py`, which writes `dist\index.html`.
3. Test: syntax checks, screenshots at the same dive moments as before, check for page errors.
4. Show Hasib before and after. Only if it looks better and nothing broke:
5. Commit, with a message that says what changed and why: `git add -A` then `git commit`.
6. Tag it with the next version number: `git tag -a v1.1 -m "..."`
7. Add a row to `COMMITTS` in `docs\make_git_tracking.py`, run it, and commit the new PDF.

If a change looks worse or breaks something, it is not committed; the files are put back with `git restore`. If a problem is found only later, that one commit is undone with `git revert` (below), and the table shows it as "Undone".

5. How to look back and go back

Type these in **Git Bash** (Start menu, "Git Bash") or in PowerShell, after moving into the project folder: `cd C:\Users\hasib\Downloads\deep-drift-project`. Or ask Claude to do it.

<code>git log --oneline --decorate</code>	List every saved version, newest first, with IDs and tags.
<code>git status</code>	Show which files changed since the last save. Safe, changes nothing.
<code>git diff</code>	Show the exact lines changed since the last save. Safe.
<code>git show v1.0:dist/index.html > old.html</code>	Get the playable page of any version as a separate file, without touching the project. Open <code>old.html</code> in a browser.

```
git switch --detach v1.0  
then git switch main
```

Visit an old version: the folder turns into v1.0. The second command comes back to the newest. Nothing is lost. (Save or undo current edits first; git will warn you.)

```
git restore src/world.js
```

Throw away unsaved edits to one file, back to the last commit.

```
git revert 1a2b3c4
```

Undo one commit by adding a new commit that does the opposite. Later changes are kept, and the history still shows what happened. **The safe way to undo.**

Careful: `git reset --hard v1.0` makes the project exactly v1.0 and **deletes unsaved edits and every commit after it** (tagged versions can still be recovered, others are hard to find). Use `git revert` or `git switch` instead, or ask Claude first.

6. The table of saved versions

Every commit, oldest first. The commit ID links a row to git: `git show 7929f37` shows everything in that commit.

#	Version	Commit ID	Date	Commit name	Detailed description	Status
1	v1.0	7929f37	24 Sep 2026, 18:22	Save the dive as it was before the realism work v1.0: save the dive as it was before the realism work	What changed: Nothing in the dive itself was changed. The whole project was put under git for the first time, so every later change can be undone. Files: • All of <code>src/</code>, <code>assets/</code>, <code>tests/</code>, <code>tools/</code>, <code>README.md</code>, <code>CLAUDE.md</code>, <code>build.py</code>, <code>.gitignore</code> • <code>dist/index.html</code> : the built, playable page • <code>package.json</code> : version changed from 0.4.0 to 1.0.0 • New: <code>backup/deep-drift-v1.0.html</code> , a plain copy of the page that opens without git • New: <code>.gitattributes</code> , keeps line endings exactly as saved State of the dive: Six places: reef, open ocean, midnight, vents, wreck, trench. One real 3D model (the barramundi fish); everything else is built by code. The weak spots listed in <code>CLAUDE.md</code> (hazy reef wall, dark wreck and trench, small deep animals, corals you can swim through, untested on phones) are all still present. Checks: All five scripts pass a syntax check, alone and joined. <code>dist/index.html</code> is identical to a fresh build. No screenshots yet: the test tools were not installed at this point.	Kept
2	none	Added at the next update (see section 3)	this commit	Add this git tracking guide docs: add the git tracking guide	What changed: Added this PDF and the small script that makes it. The dive was not changed, so this commit gets no version number. Files: • New: <code>docs/git-tracking.pdf</code> (this file) • New: <code>docs/make_git_tracking.py</code> (writes this file; holds the table entries) State of the dive: Same as v1.0. Checks: PDF opened and read through.	Kept

Planned (not saved yet)

Order proposed on 24 Sep 2026, not yet confirmed by Hasib. Rows move into the table above when they are committed. The version numbers may shift if the order changes.

Version	Change	What it will do
none	Test fixes	Make the screenshot tests work on Windows (they save to /tmp), make the strict page policy actually apply in tests, and make build.py read and write files as UTF-8. Tools only, the dive does not change.
v1.1	Reef wall	Real rock pattern on the wall, ledges and overhangs, more coral and sponges on the wall, less haze close to the wall.
v1.2	Wreck	Proper ship hull (keel, flat sides, deck, pointed bow), rails, broken planks, a hole, rust, growth on the hull, better torch light.
v1.3	Trench	Layered rock, fallen boulders, sea cucumbers and shrimp, a faint glow so it is not pure black.
v1.4	Deep animals	Thicker, brighter tube worm clusters, brighter anglerfish lure, snailfish and dumbo octopus passing closer to the camera.
v1.5	Solid corals	The diver can no longer swim through tall corals.

7. Words used in this guide

Repository (repo)	The project folder plus its <code>.git</code> history.
Commit	One saved snapshot of the whole project, with a message.
Commit ID (hash)	The fingerprint name of a commit, like <code>7929f37</code> .
Stage / staging area	The waiting list of changes for the next commit.
Branch	A moving name for the newest commit of a line of work. Ours is <code>main</code> .
Tag	A fixed name for one commit, used for version numbers.
HEAD	The commit the working folder is currently based on.
Revert	Undo a commit by adding its opposite as a new commit.
Remote	A copy of the repository somewhere else, like GitHub. We have none yet.